Alquiler de Vehículos — Diagrama de Clases UML (versión final) · Lenguajes 2 · Grupo 25

vista

Main
+ main(args: String[]): void
- cargarAgencia(gestor: GestorPersistencia): Agencia
- guardarAlCerrar(ventana, gestor, agencia): void

VentanaPrincipal
+ VentanaPrincipal(servicioFlota, servicioClientes, servicioAlquiler)

Main —crea→ VentanaPrincipal

PanelFlota
- TODOS: String
- padre: Frame
- servicio: ServicioFlota
- modelo: ModeloTablaFlota
- comboTipo: JComboBox<String>
- comboEstado: JComboBox<String>
- tablaFlota: JTable
- darDeBaja: JButton

+ PanelFlota(padre: Frame, servicio: ServicioFlota)
+ refrescar(): void
- darDeBajaElegido(): void

PanelClientes
- padre: Frame
- servicio: ServicioClientes
- modelo: ModeloTablaClientes
- campoBusqueda: JTextField
- tablaClientes: JTable
- agregarCliente: JButton
- eliminarClienteBoton: JButton

+ PanelClientes(padre: Frame, servicio: ServicioClientes)
~ alCambiarElTexto(campo: JTextField, accion: Runnable): void
- eliminarClienteElegido(): void

PanelContratos
- TODOS: String
- padre: Frame
- servicioAlquiler: ServicioAlquiler
- servicioClientes: ServicioClientes
- servicioFlota: ServicioFlota
- alCambiarFlota: Runnable
- modelo: ModeloTablaContratos
- comboEstado: JComboBox<String>
- tablaContratos: JTable
- nuevoContrato: JButton
- devolucionBoton: JButton

+ PanelContratos(padre, servicioAlquiler, servicioClientes, servicioFlota, alCambiarFlota)

PanelFlota —abre→ FormularioAuto, FormularioMoto, FormularioCamioneta
PanelClientes —abre→ FormularioCliente
PanelContratos —abre→ FormularioContrato, FormularioDevolucion

FormularioAuto
- servicio: ServicioFlota
- alRegistrar: Runnable

+ FormularioAuto(padre, servicio, alRegistrar)
- guardar(): void

FormularioMoto
- servicio: ServicioFlota
- alRegistrar: Runnable

+ FormularioMoto(padre, servicio, alRegistrar)
- guardar(): void

FormularioCamioneta
- servicio: ServicioFlota
- alRegistrar: Runnable
- capacidadCargaKg: JTextField
- traccion4x4: JCheckBox

+ FormularioCamioneta(padre, servicio, alRegistrar)
- guardar(): void

FormularioCliente
- servicio: ServicioClientes
- alRegistrar: Runnable
- nombre: JTextField
- apellido: JTextField
- dni: JTextField
- fechaNacimiento: JTextField
- telefono: JTextField
- numero: JTextField
- fechaVencimiento: JTextField
- categorias: Map<CategoriaLicencia, JCheckBox>

+ FormularioCliente(padre, servicio, alRegistrar)
- agregar(panel: JPanel, etiqueta: String, campo: Component): void
- guardar(): void
- avisarSiNoPuedeAlquilar(cliente: Cliente): void
~ fecha(etiqueta: String, campo: JTextField): LocalDate

FormularioContrato
- SIN_TOTAL: String
- servicioAlquiler: ServicioAlquiler
- alCrear: Runnable
- comboCliente: JComboBox<Cliente>
- comboVehiculo: JComboBox<Vehiculo>
- campoFechaFin: JTextField
- etiquetaTotal: JLabel

+ FormularioContrato(padre, servicioAlquiler, servicioClientes, servicioFlota, alCrear)
- guardar(): void
- actualizarTotal(): void

FormularioDevolucion
- servicioAlquiler: ServicioAlquiler
- contrato: Contrato
- alRegistrar: Runnable
- campoFechaDevolucion: JTextField
- campoKmFinal: JTextField

+ FormularioDevolucion(padre, servicioAlquiler, contrato, alRegistrar)
- guardar(): void

PanelDatosVehiculo
- patente: JTextField
- marca: JTextField
- modelo: JTextField
- color: JTextField
- anio: JTextField
- kilometraje: JTextField
- precioBaseDiario: JTextField

+ agregar(etiqueta: String, campo: Component): void
+ getPatente(): String
+ getMarca(): String
+ getModelo(): String
+ getColor(): String
+ getAnio(): int
+ getKilometraje(): int
+ getPrecioBaseDiario(): double
~ entero(etiqueta: String, campo: JTextField): int
~ decimal(etiqueta: String, campo: JTextField): double

ModeloTablaFlota
- COLUMNAS: String[]
- servicio: ServicioFlota
- filtroTipo: String
- filtroEstado: EstadoVehiculo

+ setFiltro(tipo: String, estado: EstadoVehiculo): void
+ getVehiculoEn(fila: int): Vehiculo
+ getRowCount(): int
+ getColumnCount(): int
+ getColumnName(columna: int): String
+ getValueAt(fila: int, columna: int): Object

ModeloTablaClientes
- COLUMNAS: String[]
- servicio: ServicioClientes
- filtroTexto: String

+ setFiltro(texto: String): void
+ getClienteEn(fila: int): Cliente
+ getRowCount(): int
+ getColumnCount(): int
+ getColumnName(columna: int): String
+ getValueAt(fila: int, columna: int): Object

ModeloTablaContratos
- COLUMNAS: String[]
- servicio: ServicioAlquiler
- filtroEstado: EstadoContrato

+ ModeloTablaContratos(servicio: ServicioAlquiler)
+ setFiltro(estado: EstadoContrato): void
+ getRowCount(): int
+ getColumnCount(): int
+ getColumnName(columna: int): String
+ getValueAt(fila: int, columna: int): Object
+ getContratoEn(fila: int): Contrato

servicio

ServicioFlota
- agencia: Agencia

+ registrarVehiculo(v: Vehiculo): void
+ listarVehiculos(): List<Vehiculo>
+ eliminarVehiculo(v: Vehiculo): void
+ filtrar(tipo: String, estado: EstadoVehiculo): List<Vehiculo>

ServicioClientes
- agencia: Agencia

+ registrarCliente(c: Cliente): void
- exigir(valor: String, campo: String): void
+ eliminarCliente(c: Cliente): void
+ buscarPorTexto(texto: String): List<Cliente>
+ listarClientes(): List<Cliente>

ServicioAlquiler
- agencia: Agencia

+ ServicioAlquiler(agencia: Agencia)
+ crearContrato(c, v, t, fin): Contrato
+ registrarDevolucion(c, fecha, kmFinal): void
- motivoDeRechazo(c: Cliente, v: Vehiculo, fin: LocalDate): String
+ listarContratos(estado: EstadoContrato): List<Contrato>

modelo

Formatos
+ FECHA: DateTimeFormatter

- Formatos()

Agencia
- serialVersionUID: long
- vehiculos: List<Vehiculo>
- clientes: List<Cliente>
- contratos: List<Contrato>
- ultimoIdCliente: int
- ultimoNroContrato: int

+ getVehiculos(): List<Vehiculo>
+ getClientes(): List<Cliente>
+ getContratos(): List<Contrato>
+ siguienteIdCliente(): int
+ siguienteNroContrato(): int

persistencia

GestorPersistencia
- archivo: File

+ GestorPersistencia(archivo: File)
+ guardar(a: Agencia): void
+ cargar(): Agencia
+ existeArchivo(): boolean

GestorPersistencia —serializa→ Agencia

modelo.flota

«abstract» *Vehiculo*
- serialVersionUID: long
- patente: String
- marca: String
- modelo: String
- color: String
- anio: int
- kilometraje: int
- precioBaseDiario: double
- estado: EstadoVehiculo

+ *getTipo(): String*
getFactorTarifa(): double
+ getPrecioDiario(): double
+ validarDatosPropios(): void
+ *getLicenciaRequerida(): CategoriaLicencia*
+ estaDisponible(): boolean
+ toString(): String
+ cambiarEstado(e: EstadoVehiculo): void
+ actualizarKilometraje(km: int): void

«enumeration» EstadoVehiculo
DISPONIBLE
ALQUILADO

Auto
- cantidadPuertas: int
- capacidadPasajeros: int
- tipoCaja: TipoCaja

+ getTipo(): String
getFactorTarifa(): double
+ validarDatosPropios(): void
+ getLicenciaRequerida(): CategoriaLicencia

Moto
- cilindrada: int
- incluyeCasco: boolean

+ getTipo(): String
getFactorTarifa(): double
+ validarDatosPropios(): void
+ getLicenciaRequerida(): CategoriaLicencia

Camioneta
- capacidadCargaKg: double
- traccion4x4: boolean

+ getTipo(): String
getFactorTarifa(): double
+ validarDatosPropios(): void
+ getLicenciaRequerida(): CategoriaLicencia

«enumeration» TipoCaja
MANUAL
AUTOMATICA

modelo.clientes

Cliente
- serialVersionUID: long
- id: int
- nombre: String
- apellido: String
- dni: String
- fechaNacimiento: LocalDate
- telefono: String
- licencia: LicenciaConducir
+ EDAD_MINIMA: int = 18

+ setId(id: int): void
+ getNombreCompleto(): String
+ getEdad(): int
+ toString(): String
+ puedeConducir(v: Vehiculo, hasta: LocalDate): boolean

LicenciaConducir
- serialVersionUID: long
- numero: String
- categorias: Set<CategoriaLicencia>
- fechaVencimiento: LocalDate

+ estaVigente(fecha: LocalDate): boolean
+ habilitaPara(c: CategoriaLicencia, fecha: LocalDate): boolean

«enumeration» CategoriaLicencia
A
B
C
D
E

modelo.alquiler

Contrato
- serialVersionUID: long
- numero: int
- cliente: Cliente
- vehiculo: Vehiculo
- tarifa: Tarifa
- fechaInicio: LocalDate
- fechaFinPrevista: LocalDate
- fechaDevolucion: LocalDate
- kmInicial: int
- kmFinal: int
- estado: EstadoContrato

+ Contrato(cliente, vehiculo, tarifa, fechaInicio, fechaFinPrevista)
+ setNumero(numero: int): void
+ getDias(): long
+ getKmRecorridos(): int
+ calcularTotal(): double
+ finalizar(fecha: LocalDate, kmFinal: int): void
+ getFechaDevolucion(): LocalDate
+ getKmFinal(): int

«enumeration» EstadoContrato
ACTIVO
FINALIZADO

excepciones

«java.lang» Exception

AlquilerException
+ AlquilerException(mensaje: String)
+ AlquilerException(mensaje: String, causa: Throwable)

VehiculoNoDisponibleException
+ VehiculoNoDisponibleException(patente: String)

ConductorNoHabilitadoException
+ ConductorNoHabilitadoException(motivo: String)

modelo.tarifas (patrón Strategy)

«interface» Tarifa
+ calcular(v: Vehiculo, dias: long, km: int): double
+ getNombre(): String

TarifaDiaria
- serialVersionUID: long

+ calcular(v: Vehiculo, dias: long, km: int): double
+ getNombre(): String

Contrato —tarifa→ Tarifa

Referencias
▷ línea continua: herencia · ▷ punteada: implementa interfaz
◆ composición · ◇ agregación · → asociación · ⇢ gris punteada: dependencia («crea», «abre», «serializa»)
cursiva: abstracto · subrayado: static · + público · - privado · # protegido · ~ paquete
Firmas largas: solo los nombres de los parámetros (tipos completos en uml.md).